МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-12

Выполнил

С. Э. Михновец,

студент группы ПО-12

Проверила

А. Д. Кулик,

преп. стаж. кафедры ИИТ,

«___k _____2026 г.

Брест 2026

Цель работы: научиться разрабатывать простейшие программы на языке программирования Python.

1. Задание по лабораторной работе

Задание 1: Shopping Cart

Реализовать тесты для корзины покупок (Cart), включая:

- Проверку добавления товаров и подсчета суммы.
- Использование фикстуры empty_cart.
- Параметризацию тестов для проверки скидок.
- Использование unittest.mock для симуляции HTTP-запросов (requests.post).
- Использование monkeypatch для тестирования системы купонов.

Задание 2: Тесты для ЛР №1

Написать тесты для функций, реализованных в первой лабораторной работе:

- Работа со списками (сравнение, добавление, удаление).
- Алгоритм Two Sum.
- Обработка исключений и граничных случаев.

Задание 3: TDD и поиск подстроки (Вариант 6)

Реализовать метод common(str1, str2) для поиска наибольшей общей части строк по
заданной спецификации, предварительно написав тесты.

2. Спецификации ввода-вывода

- Cart.add_item: Ввод: name (str), price (float). Выход: None. Ошибка при цене < 0.
- apply_coupon: Ввод: объект Cart, code (str). Выход: применение скидки.
- common(str1, str2): Ввод: две строки (или None). Выход: строка (общая часть). Ошибка TypeError при None.

- isequal(list): Ввод: список чисел. Выход: строка "равны", "неравны" или
"меньше двух."

3. Текст программы

3.1. Задание 1: Модуль Shopping и его тесты

shopping.py

```
"""Модуль для управления корзиной покупок."""
from typing import Dict
import requests


class Cart:
    """Класс корзины покупок."""

    def __init__(self):
        self.items = []

    def add_item(self, name: str, price: float):
        """Добавляет товар в корзину."""
        if price < 0:
            raise ValueError("Цена не может быть отрицательной")
        self.items.append({"name": name, "price": price})

    def total(self) -> float:
        """Возвращает общую стоимость товаров."""
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount: float):
        """Применяет скидку в процентах."""
        if not 0 <= discount <= 100:
            raise ValueError("Скидка должна быть от 0 до 100")
        for item in self.items:
            item["price"] *= (100 - discount) / 100


def log_purchase(item: dict):
    """Логирует покупку через POST запрос."""
    requests.post("https://example.com/log", json=item, timeout=5)
```

```
def apply_coupon(cart: Cart, coupon_code: str):
    """Применяет купон к корзине."""
    coupons: Dict[str, float] = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

test_cart.py

```
"""Тесты для модуля shopping."""
from unittest.mock import patch
import pytest
from shopping import Cart, log_purchase, apply_coupon


@pytest.fixture
def empty_cart():
    """Фикстура пустой корзины."""
    return Cart()


def test_add_item(empty_cart):  # pylint: disable=redefined-outer-name
    """Тест добавления товара."""
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1


@pytest.mark.parametrize(
    "discount, expected", [(0, 100), (50, 50), (100, 0)]
)
def test_discount(empty_cart, discount, expected):  # pylint:
disable=redefined-outer-name
    """Тест скидок."""
    empty_cart.add_item("Item", 100.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.total() == expected


@patch("requests.post")
```

```
def test_log_purchase(mock_post):
    """Тест мока запроса."""
    log_purchase({"item": "test"})
    mock_post.assert_called_once()


def test_apply_coupon(empty_cart):  # pylint: disable=redefined-outer-name
    """Тест купонов."""
    empty_cart.add_item("Item", 100.0)
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.total() == 90.0
```

3.2. Задание 2: Тесты для ЛР №1 (Списки и Two Sum)

test_lab1.py

```
"""Тестирование функций работы со списками и алгоритма Two Sum."""
from unittest.mock import patch
from reports.Mihnovec_S_E.Lab_1.src.lab1 import isequal, add_item
from reports.Mihnovec_S_E.Lab_1.src.lab1_2 import twosum


def test_isequal_logic():
    """Проверка логики сравнения элементов."""
    assert isequal([5, 5]) == "равны"
    assert isequal([1, 2]) == "неравны"
    assert isequal([1]) == "меньше двух."


def test_add_item_mock():
    """Тест добавления элемента через мок ввода."""
    items = []
    with patch("builtins.input", return_value="5"):
        add_item(items)
    assert items == [5]


def test_twosum_basic():
    """Проверка алгоритма Two Sum."""
    assert twosum(9, [2, 7, 11, 15]) == [0, 1]
```

```
    assert twosum(10, [1, 2, 3]) == []
```

3.3. Задание 3: Реализация Common Substring

string_utils.py

```
"""Модуль поиска общей подстроки."""
from typing import Optional


def common(str1: Optional[str], str2: Optional[str]) -> str:
    """Находит наибольшую общую подстроку."""
    if str1 is None or str2 is None:
        raise TypeError("Аргументы не могут быть None")
    if not str1 or not str2:
        return ""
    longest = ""
    for i in range(len(str1)):
        for j in range(i + 1, len(str1) + 1):
            sub = str1[i:j]
            if sub in str2 and len(sub) > len(longest):
                longest = sub
    return longest
```

test_string_utils.py

```
"""Тесты для функции common по спецификации."""
import pytest
from string_utils import common


def test_common_spec():
    """Проверка всех граничных случаев."""
    with pytest.raises(TypeError):
        common(None, None)
    assert common("", "") == ""
    assert common(" abc ", "abc") == "abc"
    assert common("abcde", "xyz") == ""
    assert common(" deabc ", " abcdeabcd ") == " deabc "
    assert common(" dfabcegt ", " rtoefabceiq ") == " fabce "
```

4. Выводы по проделанной работе

В ходе лабораторной работы были освоены:

1. Инструменты тестирования: Пакет pytest для автоматизации проверок.
2. Изоляция кода: Использование unittest.mock.patch для имитации ввода
   пользователя (input) и сетевых запросов (requests.post), что позволяет тестировать функции без участия человека и внешних сервисов.
3. Гибкость тестов: Фикстуры позволили избежать дублирования кода инициализации
   объектов, а параметризация позволила одной функцией проверить множество
   входных значений.
4. Стандарты качества: Весь код отформатирован инструментом Black и проверен
   линтером Pylint. Использование аннотаций типов (Optional, list[int]) предотвратило ошибки статического анализа при работе с None.
5. TDD: На примере задания №3 отработан цикл "Тест -> Реализация", что
   гарантирует соответствие кода спецификации.

Рисунки с результатами работы программы

```
========================================= test session starts =========================================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\SPP\repo\spp_po12v15
plugins: anyio-4.12.1
collected 5 items

reports\Mihnovec_S_E\Lab_6\src\lab6_1_2.py .....                                            [100%]

========================================== 5 passed in 0.06s ==========================================
PS D:\SPP\repo\spp_po12v15> python -m pytest reports/Mihnovec_S_E/Lab_6/src/test_cart.py
========================================= test session starts =========================================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\SPP\repo\spp_po12v15
plugins: anyio-4.12.1
collected 11 items

reports\Mihnovec_S_E\Lab_6\src\test_cart.py ...........                                      [100%]

========================================= 11 passed in 0.69s ==========================================
PS D:\SPP\repo\spp_po12v15> python -m pytest reports/Mihnovec_S_E/Lab_6/src/common_test.py
========================================= test session starts =========================================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\SPP\repo\spp_po12v15
plugins: anyio-4.12.1
collected 1 item

reports\Mihnovec_S_E\Lab_6\src\common_test.py .                                              [100%]

========================================= 1 passed in 0.04s ===========================================
```

Вывод: научился разрабатывать простейшие программы на Python.